

**ШИНЭ МОНГОЛ ТЕХНОЛОГИЙН КОЛЛЕЖ**

**КОМПЬЮТЕРИЙН УХААНЫ ТАНХИМ**

Оюутны код: s21c032b

Оюутны овог нэр: Эрдэнэ Ану

**ХЭРЭГЛЭГЧИЙН ДИЖИТАЛ УХААЛАГ ТЭМДЭГЛЭЛИЙН ДЭВТРИЙН АПП**

**ХӨГЖҮҮЛЭЛТ**

**/ДЭД БАКАЛАВРЫН ТӨГСӨЛТИЙН СУДАЛГААНЫ АЖИЛ/**

Удирдагч багш:

/У. Анужин, Бакалавр/

Гүйцэтгэсэн оюутан:

/s21c032b, Э. Ану/

Улаанбаатар хот

2026 он

ШИНЭ МОНГОЛ ТЕХНОЛОГИЙН КОЛЛЕЖ  
КОМПЬЮТЕРИЙН УХААНЫ ТАНХИМ

Төгсөлтийн судалгааны ажил (071309)

ХЭРЭГЛЭГЧИЙН ДИЖИТАЛ УХААЛАГ ТЭМДЭГЛЭЛИЙН ДЭВТРИЙН АПП  
ХӨГЖҮҮЛЭЛТ

Гүйцэтгэгч: Э. Ану

Удирдагч: У. Анужин, Бакалавр

Улаанбаатар хот

2026 он

# Хураангуй

Энэхүү судалгааны ажил нь хиймэл оюун ухааны технологийг ашиглан хэрэглэгчийн тэмдэглэлийн агуулгад тохирсон автомат стикер үүсгэж, тэдгээр стикерт тулгуурласан сануулга болон мэдэгдлийн системийг бүрдүүлсэн дижитал ухаалаг тэмдэглэлийн дэвтрийн аппликейшнийг боловсруулахад чиглэсэн.

Системийн backend хэсгийг **Node.js + Express.js** ашиглан хөгжүүлж, **MongoDB Atlas** үүлэн өгөгдлийн санд холбосон. Хэрэглэгчийн баталгаажуулалтыг **JWT** болон **bcrypt** технологиор хэрэгжүүлсэн. Frontend хэсгийг **Flutter (Dart)** хэлээр бичиж, Android болон iOS платформд ажиллах мобайл аппликейшн бүтээв. Тэмдэглэлийн CRUD үйлдлүүд, mood tracking функц, **flutter\_secure\_storage** ашиглан токен аюулгүй хадгалах системийг нэвтрүүллээ.

Туршилтын явцад нэвтрэлт, бүртгэл, тэмдэглэл үүсгэх/засах/устгах үйлдлүүд амжилттай ажиллаж байсан бөгөөд API хариу хугацаа дундажаар 80–150 мс байв.

**Түлхүүр үгс:** дижитал тэмдэглэл, хиймэл оюун, Flutter, Node.js, MongoDB, JWT, REST API, CRUD

## Abstract (English)

This research focuses on developing a digital smart note-taking mobile application that automatically generates AI-powered stickers based on note content, and delivers reminder notifications utilizing those stickers.

The backend was built with **Node.js + Express.js** connected to **MongoDB Atlas** cloud database. User authentication was implemented using **JWT** and **bcrypt**. The frontend was developed in **Flutter (Dart)**, supporting both Android and iOS platforms. Core features include note CRUD operations, mood tracking, and secure token storage via **flutter\_secure\_storage**.

Testing confirmed successful operation of all authentication, note management, and API communication features. Average API response time was 80–150 ms.

**Keywords:** digital note-taking, artificial intelligence, Flutter, Node.js, MongoDB, JWT, REST API, CRUD

# Гарчиг

|                                                                      |          |
|----------------------------------------------------------------------|----------|
| <b>Хураангуй</b>                                                     | <b>i</b> |
| <b>1 Удиртгал</b>                                                    | <b>1</b> |
| 1.1 Үндэслэл, ач холбогдол                                           | 1        |
| 1.2 Судалгааны асуудал                                               | 1        |
| 1.3 Зорилго, зорилт                                                  | 1        |
| 1.4 Судалгааны объект, хамрах хүрээ                                  | 2        |
| <b>2 Судалгааны Ажлын Онолын Хэсэг, Өнөөгийн Түвшин</b>              | <b>3</b> |
| 2.1 Тэмдэглэл хөтлөлтийн түүхэн хөгжил                               | 3        |
| 2.1.1 Дижитал тэмдэглэлийн эхлэл (1990–2000)                         | 3        |
| 2.1.2 Хиймэл оюун ухаан суурилсан тэмдэглэлийн үе (2020–өнөөдөр)     | 3        |
| 2.2 Дижитал тэмдэглэлийн системийн онолын үндэс                      | 3        |
| 2.2.1 REST API архитектур                                            | 3        |
| 2.2.2 JWT (JSON Web Token) баталгаажуулалт                           | 3        |
| 2.2.3 MongoDB NoSQL өгөгдлийн сан                                    | 4        |
| 2.2.4 Flutter мобайл хөгжүүлэлтийн платформ                          | 4        |
| 2.3 Mood Tracking технологийн үндэс                                  | 4        |
| 2.4 Төстэй бүтээгдэхүүнүүдтэй харьцуулалт                            | 4        |
| 2.5 Express.js фреймворкийн онолын үндэс                             | 4        |
| 2.5.1 Middleware дарааллын үзэл баримтлал                            | 5        |
| 2.5.2 Mongoose ODM (Object Document Mapper)                          | 5        |
| 2.6 flutter_secure_storage-ийн онолын үндэс                          | 5        |
| 2.7 HTTP Client Flutter дахь ашиглалт                                | 5        |
| 2.8 CORS (Cross-Origin Resource Sharing)                             | 6        |
| 2.9 RESTful API дизайны зарчим                                       | 6        |
| <b>3 Судалгааны Арга Зүй – Системийн Ажиллах Зарчим</b>              | <b>7</b> |
| 3.1 Системийн архитектур                                             | 7        |
| 3.2 AI Sticker, Calendar болон Notification системийн ажиллах зарчим | 7        |
| 3.3 UML Диаграмууд                                                   | 7        |
| 3.3.1 Use Case Diagram                                               | 7        |
| 3.3.2 Class Diagram                                                  | 9        |
| 3.3.3 Sequence Diagram – Нэвтрэх үйлдэл                              | 9        |
| 3.3.4 Sequence Diagram – Тэмдэглэл үүсгэх                            | 10       |
| 3.4 Системийн Үндсэн Компонентүүдийн Техникийн Тайлбар               | 11       |
| 3.4.1 Backend: Node.js + Express                                     | 11       |
| 3.4.2 Frontend: Flutter Мобайл Апп                                   | 11       |
| 3.4.3 AI Компонент: OpenAI DALL-E 3                                  | 12       |
| 3.5 Backend хөгжүүлэлт (Node.js + Express.js)                        | 12       |
| 3.5.1 Серверийн үндсэн тохиргоо                                      | 12       |
| 3.5.2 Өгөгдлийн загвар (Mongoose Schema)                             | 13       |
| 3.5.3 Хэрэглэгчийн баталгаажуулалт (JWT + bcrypt)                    | 16       |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 3.5.4    | Тэмдэглэлийн CRUD хэрэгжилт . . . . .                          | 18        |
| 3.6      | Middleware болон аюулгүй байдлын систем . . . . .              | 21        |
| 3.6.1    | authMiddleware.js – JWT баталгаажуулалт . . . . .              | 21        |
| 3.6.2    | API Route-д authMiddleware ашиглалт . . . . .                  | 22        |
| 3.7      | Flutter UI хэрэгжилтийн дэлгэрэнгүй . . . . .                  | 23        |
| 3.7.1    | Апп-ын Material Theme тохиргоо . . . . .                       | 23        |
| 3.7.2    | Нэвтрэх хуудас (LoginScreen) – Дэлгэрэнгүй хэрэгжилт . . . . . | 24        |
| 3.8      | Өгөгдлийн урсгалын диаграм . . . . .                           | 27        |
| <b>4</b> | <b>Судалгааны Үр Дүн</b>                                       | <b>29</b> |
| 4.1      | Системийн хэрэгжилтийн үр дүн . . . . .                        | 29        |
| 4.2      | API Туршилтын үр дүн . . . . .                                 | 29        |
| 4.3      | Туршилтын үзүүлэлтүүд . . . . .                                | 29        |
| 4.4      | Хэрэглэгчийн туршилт . . . . .                                 | 29        |
| 4.5      | Системийн туршилтын дүгнэлт . . . . .                          | 29        |
| <b>5</b> | <b>Дүгнэлт</b>                                                 | <b>31</b> |
| 5.1      | Судалгааны гол дүгнэлт . . . . .                               | 31        |
| 5.2      | Шинэлэг байдал . . . . .                                       | 31        |
| 5.3      | Хязгаарлалт болон цаашид хийх ажил . . . . .                   | 31        |
| 5.3.1    | Одоогийн хязгаарлалтууд . . . . .                              | 31        |
| 5.3.2    | Цаашид хийх ажлын чиглэл . . . . .                             | 32        |
| 5.4      | Судалгааны ажлын арга зүйн ач холбогдол . . . . .              | 32        |
| 5.4.1    | Технологийн сонголтын үндэслэл . . . . .                       | 32        |
| 5.4.2    | Хямдрал, уян хатан байдал . . . . .                            | 32        |
| 5.5      | Хэрэглэгчийн туршилтын дэлгэрэнгүй дүн шинжилгээ . . . . .     | 32        |
| 5.5.1    | Туршилтын дэлгэрэнгүй . . . . .                                | 33        |
| 5.5.2    | Туршилтын санал хүсэлт . . . . .                               | 33        |
| <b>A</b> | <b>Хавсралт</b>                                                | <b>34</b> |
| A.1      | Монгол хэл дэх хураангуй . . . . .                             | 34        |
| A.2      | Abstract in English . . . . .                                  | 34        |
| A.3      | Бүтцийн диаграм – Фолдерийн зохион байгуулалт . . . . .        | 34        |
| A.4      | authMiddleware.js – JWT шалгах middleware . . . . .            | 35        |

# Зургийн жагсаалт

|                                                                                                                          |    |
|--------------------------------------------------------------------------------------------------------------------------|----|
| Зураг 3.1. Системийн архитектурын диаграм . . . . .                                                                      | 7  |
| Зураг 3.2. Аппликейшний бүрэн ажиллагааны урсгал: Нэвтрэлт, Тэмдэглэл, AI Sticker, Calendar, Push Notification . . . . . | 8  |
| Зураг 3.3. Системийн Use Case Diagram . . . . .                                                                          | 9  |
| Зураг 3.4. Системийн Class Diagram . . . . .                                                                             | 10 |
| Зураг 3.5. Нэвтрэх үйлдлийн Sequence Diagram . . . . .                                                                   | 10 |
| Зураг 3.6. Тэмдэглэл үүсгэх Sequence Diagram . . . . .                                                                   | 11 |
| Зураг 3.7. Апп-ын бүрэн үйл ажиллагааны урсгалын диаграм . . . . .                                                       | 28 |
| Зураг 4.1. API хариу хугацааны харьцуулалт (мс) . . . . .                                                                | 30 |
| Зураг 4.2. Хэрэглэгчийн үнэлгээний үр дүн . . . . .                                                                      | 30 |

# Хүснэгтийн жагсаалт

|     |                                                         |    |
|-----|---------------------------------------------------------|----|
| 2.1 | Mood tracking-ийн төрлүүд . . . . .                     | 4  |
| 2.2 | Төстэй бүтээгдэхүүнүүдтэй харьцуулалт . . . . .         | 4  |
| 2.3 | Mongoose ODM-ийн үндсэн аргуудын тайлбар . . . . .      | 5  |
| 2.4 | flutter_secure_storage платформын харьцуулалт . . . . . | 5  |
| 2.5 | Flutter HTTP Client аргуудын тайлбар . . . . .          | 5  |
| 2.6 | RESTful API endpoint-үүдийн жагсаалт . . . . .          | 6  |
| 3.1 | API Endpoint-үүдийн жагсаалт . . . . .                  | 21 |
| 4.1 | Системийн хэрэгжүүлэлтийн бүрдүүлийн жагсаалт . . . . . | 29 |
| 4.2 | API Туршилтын үр дүн . . . . .                          | 29 |
| 4.3 | Хэрэглэгчийн туршилтын үнэлгээ . . . . .                | 29 |

# Бүлэг 1

## Удиртгал

### 1.1 Үндэслэл, ач холбогдол

Өдөр тутмын мэдээлэл, хийх ажлын жагсаалт, төлөвлөгөөгөө хурдан тэмдэглэж хаанаас ч хандах хэрэгцээ нэмэгдэж байгаа ч одоогийн дижитал тэмдэглэлийн аппликейшнүүд хэрэглэгчийн бодит шаардлагад нийцсэн ухаалаг функц дутмаг байна. Тиймээс хиймэл оюун ашиглан тэмдэглэлийн утга агуулгад тохирсон стикер автоматаар үүсгэж, тэдгээр стикерийг календарь дээр тухайн өдөрт нь байрлуулан, автоматаар мэдэгдэл илгээдэг шинэ төрлийн ухаалаг тэмдэглэлийн аппликейшн хөгжүүлэх шаардлага бий болж байна.

Жишээлбэл, хэрэглэгч “3 сарын 8-нд Sunday уулзалт байна” гэж тэмдэглэл оруулахад систем тухайн агуулгад тохирсон стикерийг хиймэл оюун ашиглан автоматаар үүсгэж, тэр стикерийг календарийн 3 сарын 8-ны нүдэнд байрлуулна. Тохируулсан цагт нь push notification илгээх эсэхийг систем өөрөө шийдэж, хэрэглэгчид сануулга хүргэнэ. Энэ нь хэрэглэгчийн мэдээллийг илүү ойлгомжтой, хурдан сэргээн санах боломжтой болгоно.

### 1.2 Судалгааны асуудал

Одоогийн дижитал тэмдэглэлийн системүүд нь тэмдэглэл хадгалах, засах, хайх үндсэн функцтэй ч дараах асуудлуудтай байна:

- Хэрэглэгчийн сэтгэл хөдлөл, mood-ийг харгалзан үздэггүй
- Тэмдэглэлийн агуулгад тохирсон визуал стикер автоматаар үүсдэггүй
- Үүсгэсэн стикерийг календарийн тодорхой өдөрт холбох боломжгүй
- Тэмдэглэлийн агуулгад суурилсан ухаалаг сануулгын систем байдаггүй
- Монгол хэрэглэгчдэд тохирсон орон нутгийн шийдэл дутмаг

### 1.3 Зорилго, зорилт

Энэхүү судалгааны ажлын үндсэн зорилго нь хиймэл оюун ухааны технологийг ашиглан хэрэглэгчийн тэмдэглэлийн агуулга болон mood-д нийцсэн стикер дизайн автоматаар үүсгэж, тэдгээр стикерийг **календарийн тодорхой өдөрт** байрлуулан, тухайн өдрийн сануулга, мэдэгдлийн системийг систем өөрөө удирдаж ажиллуулдаг дижитал ухаалаг тэмдэглэлийн дэвтрийн аппликейшныг боловсруулахад оршино.

Судалгааны зорилтуудыг дараах байдлаар тодорхойлов:

1. Дижитал тэмдэглэлийн аппликейшний хэрэглэгчийн хэрэгцээ, шаардлагыг судалж тодорхойлох.
2. Flutter (Dart) ашиглан хэрэглэгчдэд ойлгомжтой, хялбар UI/UX загвар боловсруулах.
3. Тэмдэглэл үүсгэх, засварлах, устгах, харах үндсэн CRUD үйлдлүүдийг хэрэгжүүлэх.
4. Хэрэглэгчийн сэтгэлийн байдал (mood) дээр үндэслэн тэмдэглэлийг ангилах.
5. **Хиймэл оюун ашиглан тэмдэглэлийн агуулгад тохирсон стикер дизайн автомат үүсгэх.**
6. **Стикерийг календарийн тодорхой өдөрт холбож харуулах calendar view хэрэгжүүлэх.**
7. **Тухайн өдрийн тэмдэглэлд суурилсан push notification автоматаар илгээх систем нэвтрүүлэх.**
8. Node.js болон MongoDB ашиглан өгөгдлийг хадгалах болон дамжуулах серверийн шийдэл нэвтрүүлэх.
9. JWT болон bcrypt ашиглан хэрэглэгчийн өгөгдлийн аюулгүй байдал, нууцлалыг хангах.
10. Системийн гүйцэтгэлийг тестлэж, алдааг засварлан оновчтой болгох.



## 1.4 Судалгааны объект, хамрах хүрээ

Судалгааны объект нь Шинэ Монгол Технологийн Коллежийн оюутнуудын өдөр тутмын тэмдэглэл хөтлөлтийн хэрэгцээ юм. Системийн туршилтыг коллежийн оюутнуудад гаргаж, хэрэглэгчийн санал хүсэлтийг цуглуулан үнэлгээ хийв.

### Хамрах хүрээ:

- Мобайл аппликейшн (Android болон iOS)
- REST API backend систем
- MongoDB үүлэн өгөгдлийн сан
- Хэрэглэгчийн баталгаажуулалт болон аюулгүй байдал
- Тэмдэглэлийн удирдлага (CRUD) болон mood tracking
- **AI sticker үүсгэлт** – тэмдэглэлийн агуулгад суурилсан зураг үүсгэх
- **Calendar view** – стикерийг өдөртэй холбон харуулах
- **Push notification** – тухайн өдрийн сануулга автоматаар илгээх

*This study will be conducted at New Mongol College of Technology.*

## Бүлэг 2

### Судалгааны Ажлын Онолын Хэсэг, Өнөөгийн Түвшин

#### 2.1 Тэмдэглэл хөтлөлтийн түүхэн хөгжил

Тэмдэглэл хөтлөх үйл ажиллагаа нь хүний бичгийн соёл үүссэн цагаас эхлэлтэй олон мянган жилийн түүхтэй оюуны үйл ажиллагаа юм.

##### 2.1.1 Дижитал тэмдэглэлийн эхлэл (1990–2000)

Компьютер, интернэт түгээмэл болсноор тэмдэглэл дараах хэлбэрт шилжсэн:

- Microsoft OneNote (2003) – хэвлэлийн байгууллагуудад түгээмэл
- Evernote (2008) – хоорондоо синхрончлогдох анхны систем
- Google Keep (2013) – хурдан тэмдэглэлд зориулагдсан, хөнгөн
- Notion (2016) – бүрэн workspace платформ

##### 2.1.2 Хиймэл оюун ухаан суурилсан тэмдэглэлийн үе (2020–өнөөдөр)

Сүүлийн жилүүдэд AI технологи нэмэгдсэнээр тэмдэглэлийн системүүд дараах боломжтой болсон:

- Текстийг автоматаар эмхэтгэх, хураангуй гаргах
- Тэмдэглэлийн агуулгаар зураг, sticker үүсгэх
- Sentiment analysis ашиглан хэрэглэгчийн сэтгэл хөдлөл тодорхойлох
- Автомат мэдэгдэл, сануулга илгээх

#### 2.2 Дижитал тэмдэглэлийн системийн онолын үндэс

##### 2.2.1 REST API архитектур

REST (Representational State Transfer) нь HTTP протоколд суурилсан архитектурын загвар бөгөөд client-server харилцааг тогтмолжуулдаг. REST API-ийн үндсэн зарчмууд:

- **Stateless** – сервер нь client-ийн төлөвийг хадгалдаггүй
- **Uniform Interface** – нэгдсэн endpoint бүтэц (GET, POST, PUT, DELETE)
- **Client-Server** – frontend болон backend тусдаа хөгжүүлэгддэг
- **Cacheable** – хариуг cache хийж гүйцэтгэлийг сайжруулах

##### 2.2.2 JWT (JSON Web Token) баталгаажуулалт

JWT нь хэрэглэгчийн баталгаажуулалтад ашиглагддаг стандарт токен форматыг хэлнэ [3]. Бүтэц нь 3 хэсгээс тогтоно: **Header.Payload.Signature**

- **Header** – алгоритмын мэдээлэл (HS256)
- **Payload** – хэрэглэгчийн мэдээлэл (userId, expiresIn)
- **Signature** – нууц түлхүүрээр гарын үсэг зурсан хэсэг

Манай системд JWT токеныг flutter\_secure\_storage-д аюулгүй хадгалж, дараагийн API дуудалтад Authorization: Bearer <token> header-т дамжуулна.

### 2.2.3 MongoDB NoSQL өгөгдлийн сан

MongoDB нь document-oriented NoSQL өгөгдлийн сан бөгөөд JSON-тай ижил BSON форматаар өгөгдөл хадгалдаг [1]. Давуу талууд:

- Schema-flexible – өгөгдлийн бүтцийг уян хатнаар өөрчлөх боломжтой
- Horizontal scaling – өгөгдлийг олон серверт тарааж хадгалах
- Aggregation pipeline – хүчирхэг шүүлт, нэгтгэлтийн боломж
- Atlas – үүлэн хостолт, автомат backup

### 2.2.4 Flutter мобайл хөгжүүлэлтийн платформ

Flutter нь Google-ийн гаргасан нээлттэй эхийн UI toolkit бөгөөд нэг код бичиж Android, iOS, Web, Desktop платформуудад ажиллах апп хөгжүүлэх боломжийг олгодог [5].

- **Dart хэл** – OOP, strongly-typed, AOT compiled
- **Widget tree** – UI-г widget-үүдийн мод хэлбэрээр байгуулдаг
- **Hot Reload** – кодын өөрчлөлтийг тэр даруй харах
- **Material Design** – Google-ийн дизайн стандартыг дэмжих

## 2.3 Mood Tracking технологийн үндэс

Mood tracking буюу сэтгэлийн байдлыг хянах нь хэрэглэгчийн сэтгэл хөдлөлийг тодорхойлж, тэдэнд тохирсон контент, сануулга гаргах технологи юм.

Хүснэгт 2.1. Mood tracking-ийн төрлүүд

| Mood  | Дүрлэл                   | Тайлбар                              |
|-------|--------------------------|--------------------------------------|
| Happy | Emoji: :) – аз жаргалтай | Баяр хөөртэй, эерэг сэтгэлийн байдал |
| Sad   | Emoji: :( – гунигтай     | Гунигтай, уналтын сэтгэлийн байдал   |
| Tired | Emoji: -_- – ядарсан     | Ядарсан, унтмаар байгаа байдал       |
| Angry | Emoji: >:( – уурласан    | Уурласан, хурцадмал байдал           |
| Love  | Emoji: <3 – дуртай       | Дуртай, халуун дотно байдал          |

### 2.4 Төстэй бүтээгдэхүүнүүдтэй харьцуулалт

Хүснэгт 2.2. Төстэй бүтээгдэхүүнүүдтэй харьцуулалт

| Үзүүлэлт      | Манай апп | Notion | Evernote | Google Keep |
|---------------|-----------|--------|----------|-------------|
| CRUD үйлдэл   | ✓         | ✓      | ✓        | ✓           |
| Mood tracking | ✓         | –      | –        | –           |
| JWT auth      | ✓         | ✓      | ✓        | ✓           |
| Монгол UI     | ✓         | –      | –        | –           |
| REST API      | ✓         | –      | ✓        | –           |
| Нээлттэй эх   | ✓         | –      | –        | –           |

### 2.5 Express.js фреймворкийн онолын үндэс

Express.js нь Node.js-д суурилсан хурдан, хялбар веб framework юм [4]. Middleware болон routing механизмаараа RESTful API барихад өргөн хэрэглэгддэг.

### 2.5.1 Middleware дарааллын үзэл баримтлал

Express.js-д middleware гэдэг нь HTTP хүсэлтийн дунд гүйцэтгэгддэг функц юм. Дараалал дараах байдлаар явагдана:

1. Клиентийн хүсэлт ирнэ (HTTP Request)
2. cors() – Cross-Origin аюулгүй байдлыг шалгана
3. express.json() – JSON body-г parse хийнэ
4. authMiddleware – JWT токеныг шалгана (хамгаалагдсан route-уудад)
5. Route handler – бизнесийн логик гүйцэтгэнэ
6. HTTP хариу буцаана

### 2.5.2 Mongoose ODM (Object Document Mapper)

Mongoose нь MongoDB-тэй ажиллахад schema-based шийдэл санал болгодог Node.js library юм. Schema тодорхойлж, model үүсгэж, CRUD үйлдэл хийхэд ашиглана.

Хүснэгт 2.3. Mongoose ODM-ийн үндсэн аргуудын тайлбар

| Mongoose арга            | Тайлбар                         |
|--------------------------|---------------------------------|
| Model.save()             | Шинэ document хадгалах          |
| Model.find()             | Нөхцөлийн дагуу хайх            |
| Model.findOneAndUpdate() | Нэг document олж шинэчлэх       |
| Model.findOneAndDelete() | Нэг document олж устгах         |
| Model.findOne()          | Нөхцөлд тохирсон эхний doc олох |

## 2.6 flutter\_secure\_storage-ийн онолын үндэс

flutter\_secure\_storage нь Flutter-т зориулсан аюулгүй хадгалалтын package юм. iOS-д Keychain, Android-д EncryptedSharedPreferences ашигладаг.

Хүснэгт 2.4. flutter\_secure\_storage платформын харьцуулалт

| Платформ | Технологи                  | Аюулгүй байдлын түвшин        |
|----------|----------------------------|-------------------------------|
| Android  | EncryptedSharedPreferences | AES 256-bit шифрлэлт          |
| iOS      | Keychain Services          | Hardware-level аюулгүй байдал |

Хадгалалтын үндсэн арга:

- write(key, value) – утга хадгалах
- read(key) – утга унших
- delete(key) – утга устгах
- deleteAll() – бүгдийг устгах (logout)

## 2.7 HTTP Client Flutter дахь ашиглалт

Flutter-д http package ашиглан REST API-тай харьцана. Үндсэн HTTP арга:

Хүснэгт 2.5. Flutter HTTP Client аргуудын тайлбар

| HTTP арга | Flutter код            | Зориулалт    |
|-----------|------------------------|--------------|
| GET       | http.get(uri, headers) | Өгөгдөл авах |

|        |                      |                     |
|--------|----------------------|---------------------|
| POST   | http.post(uri, body) | Шинэ өгөгдөл үүсгэх |
| PUT    | http.put(uri, body)  | Өгөгдөл шинэчлэх    |
| DELETE | http.delete(uri)     | Өгөгдөл устгах      |

## 2.8 CORS (Cross-Origin Resource Sharing)

Манай систем frontend болон backend өөр портод ажилладаг тул CORS тохиргоо зайлшгүй шаардлагатай. Backend-д cors() middleware-ийг server.js-д ашиглана.

CORS-ийн тохиргоогүй бол browser нь cross-origin хүсэлтийг хориглоно. npm install cors командаар суулгаж, app.use(cors()) дуудна.

## 2.9 RESTful API дизайны зарчим

Манай API-ийн endpoint-үүд Richardson Maturity Model-ийн 2-р түвшинд байна (HTTP арга + Resource-based URL):

Хүснэгт 2.6. RESTful API endpoint-үүдийн жагсаалт

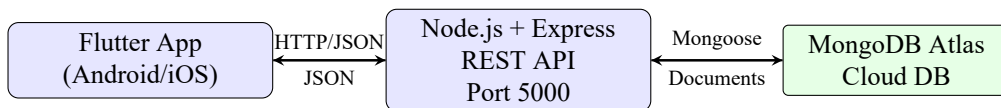
| Resource         | URL            | HTTP методууд        |
|------------------|----------------|----------------------|
| Хэрэглэгч        | /api/auth      | POST (login, signup) |
| Тэмдэглэл        | /api/notes     | GET, POST            |
| Тэмдэглэл (id-р) | /api/notes/:id | PUT, DELETE          |

## Бүлэг 3

### Судалгааны Арга Зүй – Системийн Ажиллах Зарчим

#### 3.1 Системийн архитектур

Систем нь **Client-Server** архитектурын загвараар хийгдсэн бөгөөд гурван үндсэн давхаргатай:



Зураг 3.1. Системийн архитектурын диаграм

#### 3.2 AI Sticker, Calendar болон Notification системийн ажиллах зарчим

Манай системийн гол шинэлэг нь **AI стикер үүсгэлт** → **Calendar дээр байрлуулалт** → **Автомат notification** урсгалд байна. Жишээлбэл:

“3 сарын 8-нд Sunday уулзалт байна” гэж тэмдэглэл оруулахад:

1. **Тэмдэглэл шинжлэх** – хэрэглэгч текстэн агуулгаа оруулаад
2. **AI стикер үүсгэх** – тэмдэглэлийн утга агуулга болон mood-д тохирсон стикер дизайныг хиймэл оюун автомат үүсгэнэ
3. **Calendar дээр байрлуулалт** – үүсгэсэн стикер 3 сарын 8-ны нүдэнд автоматаар холбогдоно
4. **reminderAt огноо хадгалах** – MongoDB-д Хэзэгд өгөөх цагийг хадгална
5. **Push notification илгээх** – reminderAt цаг болохд Firebase Cloud Messaging ашиглан сануулга илгээдэг [2]

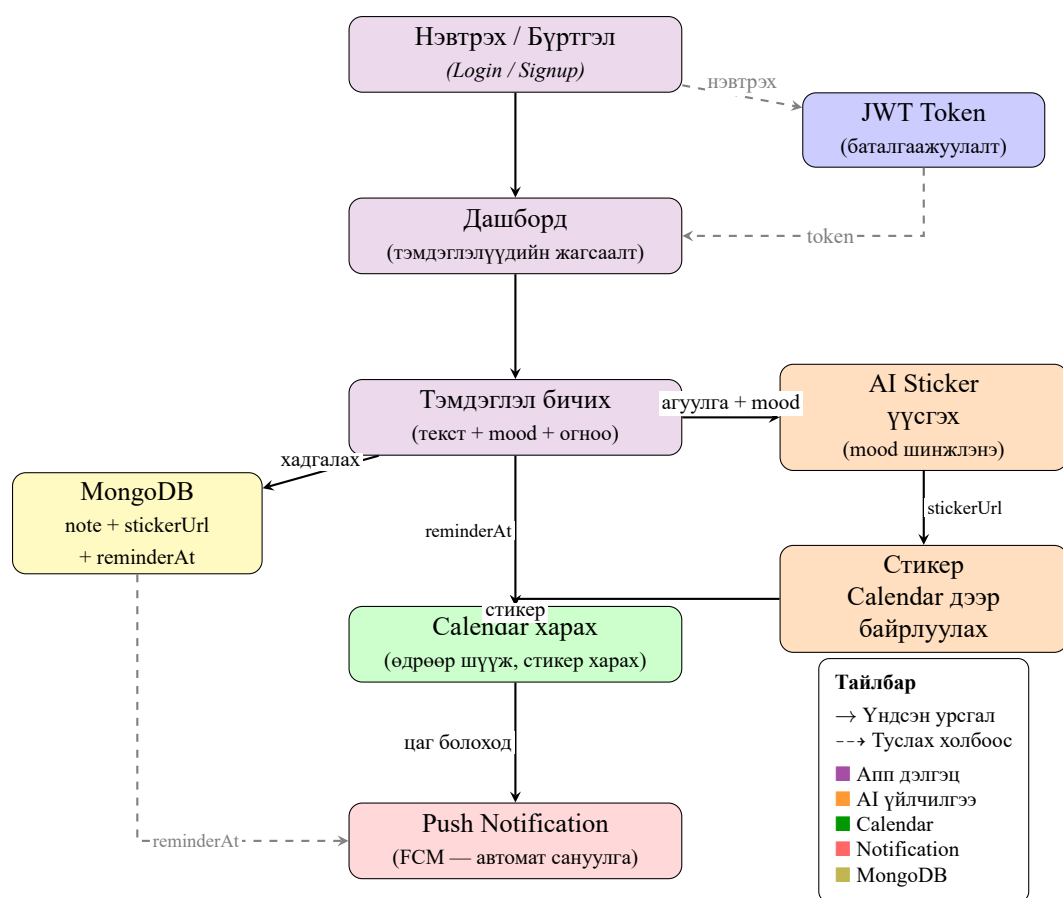
**Note өгөгдлийн загварт** reminderAt болон stickerUrl талбаруудыг нэмжээ систем дээр хадгална. Flutter календар дээр reminderAt дараах өгөгдлүүдийг шүүж, тодорхой өдөрт stickerUrl-д татасан стикерийг дүрслэн харуулна.

#### 3.3 UML Диаграмууд

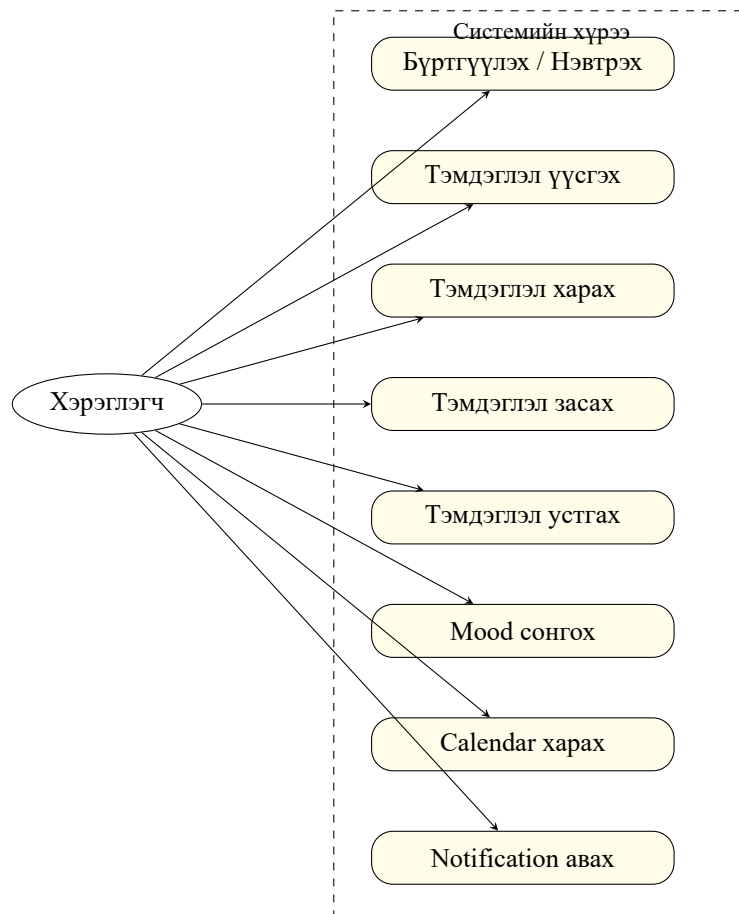
##### 3.3.1 Use Case Diagram

**Use Case Diagram** нь системд хэн ямар үйлдэл хийж болохыг харуулдаг UML диаграм юм. 3.3-р зурагт харуулснаар манай систем нэг үндсэн актортой – **Хэрэглэгч**. Тэр нь системтэй холбогддог зургаан хэрэглээний тохиолдол (use case) байна:

- **Бүртгүүлэх / Нэвтрэх** – шинэ хэрэглэгч бүртгүүлж, системд аюулгүй нэвтрэх боломж
- **Тэмдэглэл үүсгэх** – гарчиг, агуулга, mood бүхий шинэ тэмдэглэл оруулна
- **Тэмдэглэл харах** – өнөөдрийн болон бүх тэмдэглэлийн жагсаалтыг харна
- **Тэмдэглэл засах** – одоо байгаа тэмдэглэлийн агуулгыг шинэчилнэ
- **Тэмдэглэл устгах** – тэмдэглэлийг бүрмөсөн устгана
- **Mood сонгох** – Happy, Sad, Tired, Angry, Love-аас нэгийг сонгоно
- **Calendar харах** – стикер бүхий календараас өдөр сонгох
- **Notification авах** – тухайн өдрийн автоматаар сануулга хүлээнэ



Зураг 3.2. Аппликейшний бүрэн ажиллагааны урсгал: Нэвтрэлт, Тэмдэглэл, AI Sticker, Calendar, Push Notification



Зураг 3.3. Системийн Use Case Diagram

### 3.3.2 Class Diagram

**Class Diagram** нь системийн өгөгдлийн бүтэц болон классуудын хоорондын хамаарлыг харуулна. 3.4-р зурагт үзүүлснээр систем нь хоёр үндсэн классаас тогтоно:

- **User** класс – хэрэглэгчийн нэр, имэйл, нууцлагдсан нууц үг, огноо хадгална. login() арга JWT токен буцаана.
- **Note** класс – тэмдэглэлийн гарчиг, агуулга, mood, сануулгын огноо, үүсгэсэн болон шинэчилсэн огноог хадгална.

Хоёр класс нь **1-to-many** хамаарлтай: нэг хэрэглэгч олон тэмдэглэл үүсгэж болно. Note-ийн userId талбар нь User-ийн \_id-г лавлана (Mongoose ref).

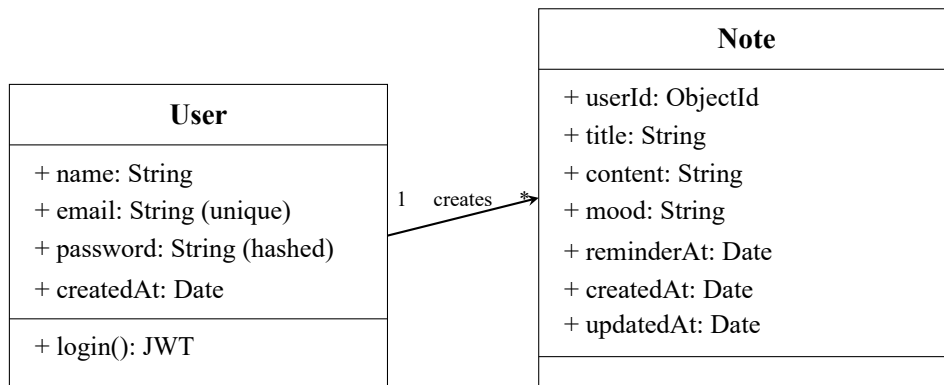
Системийн үндсэн 2 класс – **User** болон **Note**:

### 3.3.3 Sequence Diagram – Нэвтрэх үйлдэл

**Sequence Diagram** нь объектуудын хоорондын мессежийн дарааллыг цагийн хувьд харуулна. 3.5-р зурагт нэвтрэх үйл явцын бүрэн дараалал харагдана:

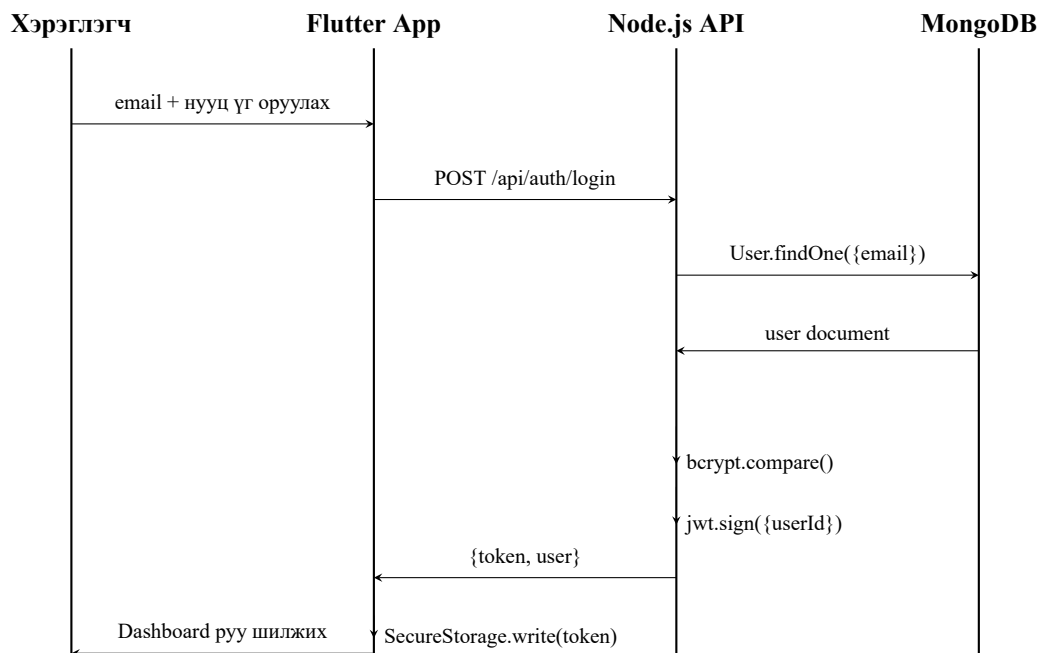
1. Хэрэглэгч имэйл болон нууц үгээ Flutter App-д оруулна
2. Flutter App нь POST /api/auth/login хүсэлт илгээнэ
3. Node.js API User.findOne({email}) ашиглан MongoDB-с хэрэглэгч хайна
4. MongoDB хэрэглэгчийн document буцаана
5. API bcrypt.compare() ашиглан нууц үгийг шалгана





Зураг 3.4. Системийн Class Diagram

6. Зөв бол `jwt.sign({userId})` ашиглан 7 хоногийн токен үүсгэнэ
7. Flutter App токеныг FlutterSecureStorage-д аюулгүй хадгална
8. Хэрэглэгчийг Dashboard руу шилжүүлнэ



Зураг 3.5. Нэвтрэх үйлдлийн Sequence Diagram

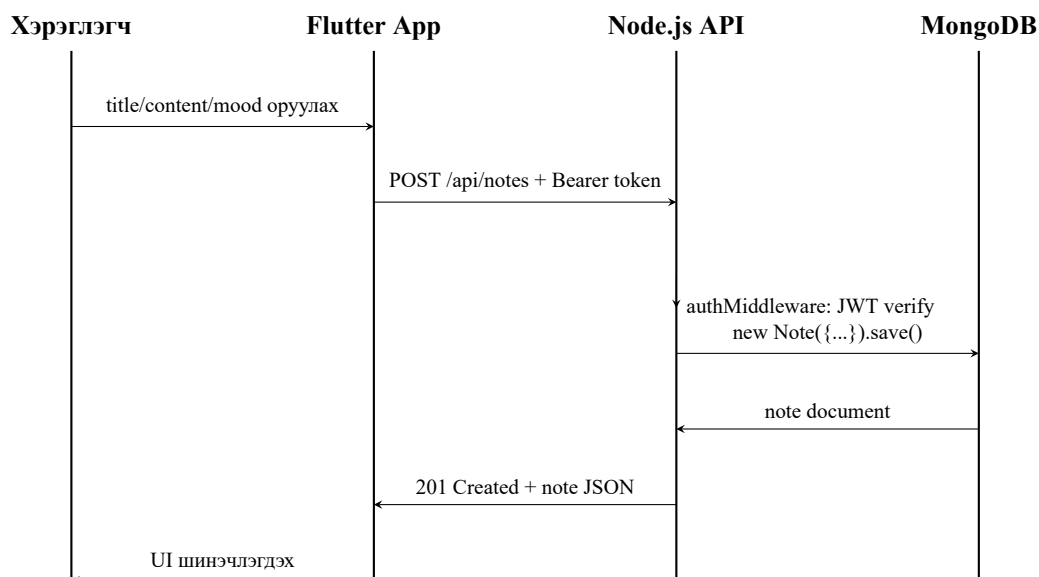
### 3.3.4 Sequence Diagram – Тэмдэглэл үүсгэх

3.6-р зурагт тэмдэглэл үүсгэх үйл явцын дараалал харагдана. Энэ үйлдэл нь заавал **JWT баталгаажуулалт** шаарддаг – токенгүй хүсэлт 401 Unauthorized буцаана:

1. Хэрэглэгч гарчиг, агуулга, mood-ыг оруулж *Хадгалах* товч дарна
2. Flutter App POST /api/notes хүсэлтийг Authorization: Bearer <token> header-тэй илгээнэ
3. authMiddleware JWT токеныг шалгаж req.userId тохируулна
4. noteController.createNote() шинэ Note объект үүсгэж MongoDB-д хадгална
5. MongoDB хадгалагдсан document буцаана

6. API 201 Created статустай note JSON буцаана

7. Flutter App UI-г шинэчилж шинэ тэмдэглэлийг жагсаалтад харуулна



Зураг 3.6. Тэмдэглэл үүсгэх Sequence Diagram

### 3.4 Системийн Үндсэн Компонентүүдийн Техникийн Тайлбар

Систем нь гурван үндсэн компонент – Backend (Node.js), Frontend (Flutter), AI (DALL-E) – -ээр бүрдэн, REST API болон JSON-ээр холбогддог модульчилсан архитектурын загвараар дизайн хийгдсэн.

#### 3.4.1 Backend: Node.js + Express

Node.js рантайм орчин дээр Express.js framework ашиглан хөгжүүлэгдсэн REST API сервер.

- **Баталгаажуулалт:** JWT token + bcrypt ашиглан нэвтрэх үйлдэл хароод хасад
- **Өгөгдлийн удирдлага:** MongoDB + Mongoose ашиглан User/Note schema удирдах
- **API эндпойнтүүд:** CRUD үйлдлүүд, JWT middleware баталгаажуулалт
- **AI интеграци:** POST /api/notes/generate-sticker ү OpenAI DALL-E API холбоход

Серверийн эргүүлэлт: Request → Middleware (CORS, JSON parse) → Authentication (JWT) → Controller (бизнес логик) → Database операци → JSON Response.

#### 3.4.2 Frontend: Flutter Мобайл Апп

Flutter framework-ээр хөгжүүлэгдсэн Android/iOS кроссплатформ мобайл апп.

- **Нэвтрэх:** FlutterSecureStorage ашиглан JWT токеныг шифрлэн хадгалах, автоматаар сессийн шалгах
- **Үндсэн экранууд:** LoginScreen → Dashboard (өдрийн тэмдэглэл) → Calendar → StickerScreen (AI)
- **UI Design:** Material Design 3, Dark mode, Purple/Pink өнгөний схем
- **Mood:** 5 төрөл (Happy, Sad, Tired, Angry, Love) сонгох боломж

Тэмдэглэл үүсгэх урсгал: Текст + mood оруулах → POST /api/notes илгээх → Backend MongoDB-д хадгалах → UI refresh.



```

44 .connect(MONGO_URI, {
45   useNewUrlParser: true ,
46   useUnifiedTopology: true ,
47 })
48 .then(() => {
49   console.log("[OK] MongoDB амжилттайхолбогдлоо !");
50 })
51 .catch((err) => {
52   console.log("[ALDAA] MongoDB холбооналдаа :", err.message);
53   process.exit(1);
54 });
55
56 // ████████████████████████████████████████████████████████████████████████████
57 // ═══════ 6. API ROUTEҮҮД- БҮРТГЭХ ═══════
58 // ████████████████████████████████████████████████████████████████████████████
59 app.use("/api/auth", authRoutes);
60 app.use("/api/notes", noteRoutes);
61
62 // ████████████████████████████████████████████████████████████████████████████
63 // ═══════ 7. HEALTH CHECK ENDPOINT ═══════
64 // ████████████████████████████████████████████████████████████████████████████
65 app.get("/", (req, res) => {
66   res.json({
67     message: "[START] AI Sticker Note API хэвийнажилландбайна ",
68     version: "1.0.0"
69   });
70 });
71
72 // ████████████████████████████████████████████████████████████████████████████
73 // ═══════ 8. СЕРВЕРАСААЛТЫГ ═══════
74 // ████████████████████████████████████████████████████████████████████████████
75 const PORT = process.env.PORT || 5000;
76 app.listen(PORT, () => {
77   console.log("\n" + "=".repeat(60));
78   console.log(`[START] Сервер ${PORT} портодэхлэлээ !`);
79   console.log("=".repeat(60));
80   console.log("[INFO] API Endpoints:");
81   console.log(`    POST    /api/auth/signup `);
82   console.log(`    POST    /api/auth/login `);
83   console.log(`    GET     /api/notes `);
84   console.log(`    POST    /api/notes `);
85   console.log(`    PUT     /api/notes/:id `);
86   console.log(`    DELETE  /api/notes/:id `);
87   console.log("=".repeat(60) + "\n");
88 });

```

### 3.5.2 Өгөгдлийн загвар (Mongoose Schema)

Listing 3.2. models/User.js – Хэрэглэгчийн схем

```

12  name: {
13      type: String,
14      required: true,
15      trim: true, // Урдхойс/ зайустгана : "  John  " → "John"
16  },
17
18  // -----
19  // email: String - Электроншуудан (Primary Key)
20  // unique: true →Ижилимэйл 2 удааорногэмалдасурпад
21  // -----
22  email: {
23      type: String,
24      required: true,
25      unique: true, // duplicate check
26      lowercase: true, // "USER@GMAIL.COM" → "user@gmail.com"
27      trim: true,
28      match: /^[^\s@]+@[^\s@]+\.[^\s@]+$/, // Email format validation
29  },
30
31  // -----
32  // password: String - Шифрлэгдсэннууцуг (bcrypt)
33  // -----
34  password: {
35      type: String,
36      required: true,
37      minlength: 8, // аас8- доошгүйурттайбайхшаардлага
38  },
39
40  // -----
41  // createdAt: Date - Бүртгүүлэхүеийнцаг
42  // default: Date.now →автоматаародоогийнцагоноожбайна
43  // -----
44  createdAt: {
45      type: Date,
46      default: Date.now,
47  },
48  });
49
50  // =====
51  // ══ Pre-Save Hook: Нууцгүйгхашлах (bcrypt) ══
52  // =====
53  // MongoDB руухадгалахаасөмнө :
54  // "password123" → "$2a$10$k3j4h2k341jh2k3hj4k2h..." хашлагдсан()
55  userSchema.pre("save", async function (next) {
56      // Нууцгүйгөөрчлөгдөөгүйбол skip хийнэ
57      if (!this.isModified("password")) {
58          return next();
59      }
60
61      try {
62          // bcrypt.genSalt(10) - 10 хосциклхүндээслэх ( зэрэг)
63          const salt = await bcryptjs.genSalt(10);
64          // bcryptjs.hash - нууцгүйг salt ээр- хашлана
65          this.password = await bcryptjs.hash(this.password, salt);
66          next();
67      } catch (error) {
68          next(error);
69      }
70  });
71
72  module.exports = mongoose.model("User", userSchema);

```

### Listing 3.3. models/Note.js – Тэмдэглэлийн схем

```

1 // ┌───────────────────────────────────────────┐
2 //   ╰═══ Note Collection Schema ═══╯
3 // Хэрэглэгчийнтэмдэглэлийнмэдээллийгхадгалах
4 // └───────────────────────────────────────────┘
5 const mongoose = require("mongoose");
6
7 const noteSchema = new mongoose.Schema({
8   // ────────────────────────────────────────────
9   // userId: ObjectId - Альхэрэглэгчийнтэмдэглэл
10  // ref: "User" → User collection руу- холбох
11  // ────────────────────────────────────────────
12  userId: {
13    type: mongoose.Schema.Types.ObjectId ,
14    ref: "User", // User modelээр- холбох
15    required: true ,
16  },
17
18  // ────────────────────────────────────────────
19  // title: String - Тэмдэглэлийнгарчиг
20  // default: "" →хэлиүгавахгүйбайжболно
21  // ────────────────────────────────────────────
22  title: {
23    type: String ,
24    default: "",
25  },
26
27  // ────────────────────────────────────────────
28  // content: String - Тэмдэглэлийнүндсэнагуулга
29  // required: true →заавалбайх
30  // ────────────────────────────────────────────
31  content: {
32    type: String ,
33    required: true ,
34  },
35
36  // ────────────────────────────────────────────
37  // mood: String - Сэтгэлхөдлөл (Happy, Sad, Tired...)
38  // default: "" →орхиболомжтойзаавал ( биш)
39  // ────────────────────────────────────────────
40  mood: {
41    type: String ,
42    enum: ["Happy", "Sad", "Tired", "Angry", "Love", ""],
43    default: "",
44  },
45
46  // ────────────────────────────────────────────
47  // stickerUrl: String - Агаар- үүсгэгдсэнстикерийн URL
48  // default: null →хэли axios
49  // ────────────────────────────────────────────
50  stickerUrl: {
51    type: String ,
52    default: null ,
53  },
54
55  // ────────────────────────────────────────────
56  // reminderAt: Date - Сануулгынцаг
57  // Жишээ: 2024-03-10T14:30:00Z

```

```

58 // -----
59 reminderAt: {
60   type: Date,
61   default: null,
62 },
63
64 // -----
65 // createdAt: Date – Тэмдэглэлүүсгэгдсэнцаг
66 // -----
67 createdAt: {
68   type: Date,
69   default: Date.now,
70 },
71
72 // -----
73 // updatedAt: Date – Сүүлийнөөрчлөгдсөнцаг
74 // -----
75 updatedAt: {
76   type: Date,
77   default: Date.now,
78 },
79 });
80
81 module.exports = mongoose.model("Note", noteSchema);

```

### 3.5.3 Хэрэглэгчийн баталгаажуулалт (JWT + bcrypt)

Listing 3.4. routes/auth.js – Бүртгэх endpoint

```

1 // =====
2 // SIGNUP: Шинэхэрэглэгчбүртгэх
3 // =====
4 const express = require("express");
5 const jwt = require("jsonwebtoken");
6 const bcryptjs = require("bcryptjs");
7 const User = require("../models/User");
8 const JWT_SECRET = process.env.JWT_SECRET || "secret123";
9 const router = express.Router();
10
11 router.post("/signup", async (req, res) => {
12   try {
13     // =====
14     // 1. Request body- хэрэглэгчийнөгөгдлийгавна
15     // =====
16     const { name, email, password, passwordConfirm } = req.body;
17
18     // Бүхталбарбайхэсэхшалгах
19     if (!name || !email || !password || !passwordConfirm) {
20       return res.status(400).json({
21         message: "Бүх" талбарыгдүүргэнэүү "
22       });
23     }
24
25     // =====
26     // 2. Нууцүгтэнцэхэсэхшалгах
27     // =====
28     if (password !== passwordConfirm) {
29       return res.status(400).json({
30         message: "Нууц" үгдавцахгүйбайна "
31       });
32     }

```

```

33
34 // =====
35 // 3. Ижилмэйлбайхгүйэсэхшалгах (unique)
36 // =====
37 const existingUser = await User.findOne({ email });
38 if (existingUser) {
39   return res.status(400).json({
40     message: Энэ" имэйлбүртгэлтэйбайна "
41   });
42 }
43
44 // =====
45 // 4. Шинэ User объектүүсгэхнууц (үгавтомат bcrypt хашлагдана)
46 // pre("save") hookд- bcryptjs.hash() дуудагдажхашлагдана
47 // =====
48 const user = new User({
49   name: name,
50   email: email,
51   password: password,
52 });
53
54 // =====
55 // 5. MongoDBд- хадгалах
56 // user.save() дуудахад pre("save") hook ажиллана
57 // =====
58 await user.save();
59
60 res.status(201).json({
61   message: "[OK] Хэрэглэгчамжилттайбүртгэгдлээ ",
62   userId: user._id
63 });
64 } catch (error) {
65   res.status(500).json({
66     message: Алдаа": " + error.message
67   });
68 }
69 });

```

Listing 3.5. routes/auth.js – Нэвтрэх endpoint

```

1 // =====
2 // LOGIN: Нэвтэрч JWT токенавах
3 // =====
4 router.post("/login", async (req, res) => {
5   try {
6     // =====
7     // 1. Request bodyс- имэйлболоннууцүгийгавна
8     // =====
9     const { email, password } = req.body;
10
11     // Бүхталбарбайхэсэхшалгах
12     if (!email || !password) {
13       return res.status(400).json({
14         message: Имэйл" болоннууцүгийгоруулнау "
15       });
16     }
17
18     // =====
19     // 2. Emailp- MongoDBд- хэрэглэгчхайна
20     // findOne() нэгл document буцаанатэмдэглэлүүдэс (ялгаатай)
21     // =====

```



```

22     const user = await User.findOne({ email: email });
23     if (!user) {
24         return res.status(400).json({
25             message: "[ALDAA] Иймимэйлтэйхэрэглэгчбайхгүй "
26         });
27     }
28
29     // =====
30     // 3. Нууцгийг bcrypt ашигланхарьцуулна
31     // bcryptjs.compare() - сэвсэннууцуг→хашлагдсаннууцуг
32     // Үрдүн : true зөв() эсвэл false буруу()
33     // =====
34     const isMatch = await bcryptjs.compare(password, user.password);
35     if (!isMatch) {
36         return res.status(400).json({
37             message: "[ALDAA] Нууцүгбуруу "
38         });
39     }
40
41     // =====
42     // 4. JWT токенүүсгэх
43     // jwt.sign() нь Header.Payload.Signature хэлбэрээрүүсгэнэ
44     // Payloadд-: userId, expiresIn: 7 хоног
45     // =====
46     const token = jwt.sign(
47         { userId: user._id }, // Payload: user ID
48         JWT_SECRET,          // Secret key: .env файлдээрээс
49         { expiresIn: "7d" }   // 7 хоногхүчинтэй
50     );
51
52     // =====
53     // 5. Клиентэдтокен + хэрэглэгчийнмэдээллийгбуцаана
54     // =====
55     res.json({
56         message: "[ОК] Амжилттайнэвтэрлээ ",
57         token: token,
58         user: {
59             userId: user._id,
60             name: user.name,
61             email: user.email
62         }
63     });
64 } catch (error) {
65     res.status(500).json({
66         message: "Алдаа": " + error.message
67     });
68 }
69 });
70
71 module.exports = router;

```

### 3.5.4 Тэмдэглэлийн CRUD хэрэгжилт

Listing 3.6. controllers/noteController.js – Тэмдэглэл үүсгэх

```

1 // =====
2 // CREATE: Шинэтэмдэглэлүүсгэх (POST) =====
3 // =====
4 const Note = require("../models/Note");
5
6 exports.createNote = async (req, res) => {

```

```

7   try {
8     // =====
9     // 1. Request body- тэмдэглэлийнөгөгдлийгавна
10    // =====
11    const { title, content, mood, reminderAt } = req.body;
12
13    // Content заавалбайхёстой
14    if (!content) {
15      return res.status(400).json({
16        message: Агуулга" хоосонбайжболохгүй "
17      });
18    }
19
20    // =====
21    // 2. Шинэ Note объектүүсгэх
22    // req.userId → authMiddleware-с ирсэн (JWT-с задалсан)
23    // =====
24    const note = new Note({
25      userId: req.userId,      // Альхэрэглэгчийнтэмдэглэл
26      title: title || "",      // Гарчигхэл ( байжболно )
27      content: content,        // Агуулгазаавал ( )
28      mood: mood || "",        // Сэтгэлхөдлөлхэл ( байжболно )
29      reminderAt: reminderAt || null, // Сануулгынцаг
30    });
31
32    // =====
33    // 3. MongoDB-д хадгалах
34    // note.save() → MongoDB-г оруулж _id авна
35    // =====
36    await note.save();
37
38    // =====
39    // 4. 201 Created статустайхүнэлэлбуцаана
40    // =====
41    res.status(201).json({
42      message: "[ОК] Тэмдэглэламжилттайүүсгэгдлээ ",
43      note: note
44    });
45  } catch (error) {
46    res.status(500).json({
47      message: " Алдаа: " + error.message
48    });
49  }
50 };
51
52 // =====
53 // READ: Бүхтэмдэглэлавах (GET) =====
54 // =====
55 exports.getNotes = async (req, res) => {
56   try {
57     // req.userId → authMiddleware-с ирсэн
58     // find({ userId: req.userId }) → зөвхөнэнэхэрэглэгчийнтэмдэглэлүүд
59     const notes = await Note.find({ userId: req.userId })
60       .sort({ createdAt: -1 }); // Сүүлийнбичлэгийгхэндүзүүлэх
61
62     res.json({
63       message: "[ОК] Тэмдэглэлүүдавагдлаа ",
64       count: notes.length,
65       notes: notes
66     });
67   } catch (error) {

```



```

129 // =====
130 const note = await Note.findOneAndDelete({
131   _id: req.params.id,
132   userId: req.userId, // Зөвхөнөөрүүдийнхустгах
133 });
134
135 if (!note) {
136   return res.status(404).json({
137     message: "[ALDAA] Тэмдэглэл олдсонгүй "
138   });
139 }
140
141 res.json({
142   message: "[OK] Тэмдэглэл устгагдлаа ",
143   deletedNoteId: note._id
144 });
145 } catch (error) {
146   res.status(500).json({
147     message: "[ALDAA] Алдаа: " + error.message
148   });
149 }
150 };

```

## API Endpoint-үүдийн жагсаалт:

Хүснэгт 3.1. API Endpoint-үүдийн жагсаалт

| Endpoint         | Арга   | Тайлбар                 |
|------------------|--------|-------------------------|
| /api/auth/signup | POST   | Шинэ хэрэглэгч бүртгэх  |
| /api/auth/login  | POST   | Нэвтрэх, JWT токен авах |
| /api/notes       | POST   | Шинэ тэмдэглэл үүсгэх   |
| /api/notes       | GET    | Бүх тэмдэглэл харах     |
| /api/notes/:id   | PUT    | Тэмдэглэл засах         |
| /api/notes/:id   | DELETE | Тэмдэглэл устгах        |

## 3.6 Middleware болон аюулгүй байдлын систем

### 3.6.1 authMiddleware.js – JWT баталгаажуулалт

Listing 3.7. middleware/authMiddleware.js – JWT validation

```

1 // =====
2 // authMiddleware: JWT баталгаажуулалт
3 // =====
4 // Энэ middleware-ийг /api/notes доторх route-уудад ашиглан
5 // токены хүчинтэй эсэхийг баталгаажуулна
6 const jwt = require("jsonwebtoken");
7 const JWT_SECRET = process.env.JWT_SECRET || "secret123";
8
9 module.exports = (req, res, next) => {
10   try {
11     // =====
12     // 1. Authorization header-г токен авна
13     // Header хэлбэр: "Authorization: Bearer eyJhbGc..."
14     // =====
15     const authHeader = req.headers.authorization;
16
17     // Токен байхгүй бол 401 Unauthorized буцаана
18     if (!authHeader) {
19       return res.status(401).json({
20         message: "[ALDAA] Токен олдсонгүй. Authorization header шаардлагатай"

```

```

21     });
22 }
23
24 // =====
25 // 2. "Bearer <token>" хэлбэрээ токеныг задалж авна
26 // split(" ") → ["Bearer", "eyJhbGc..."]
27 // [1] → "eyJhbGc..." зөвхөн(токен)
28 // =====
29 const token = authHeader.split(" ")[1];
30
31 if (!token) {
32     return res.status(401).json({
33         message: "[ALDAA] Токены формат буруу . 'Bearer <token>' ашиглана уу "
34     });
35 }
36
37 // =====
38 // 3. JWT токеныг баталгаажуулна
39 // jwt.verify() – Signature чекк, expiration чекк
40 // Амжилттай бол decoded объекта вна : { userId, exp, iat }
41 // =====
42 const decoded = jwt.verify(token, JWT_SECRET);
43
44 // =====
45 // 4. Задалсан userId-г req.userId-д хийнэ
46 // Дараа route handler нь req.userId авч ашиглаж болно
47 // =====
48 req.userId = decoded.userId;
49
50 // =====
51 // 5. next() дуудаж дараа middleware буюу controller руу явна
52 // =====
53 next();
54 } catch (err) {
55     // Токен буруу, сүүлэгдсэн, эсвэл signature алдаа
56     if (err.name === "TokenExpiredError") {
57         return res.status(401).json({
58             message: "[ALDAA] Токеныг сүүлэгдсэн . Дахин нэвтрээрэй "
59         });
60     } else if (err.name === "JsonWebTokenError") {
61         return res.status(401).json({
62             message: "[ALDAA] Токеныг буруу . Очих боломжгүй "
63         });
64     } else {
65         return res.status(401).json({
66             message: "[ALDAA] Баталгаажуулалтын алдаа : " + err.message
67         });
68     }
69 }
70 };

```

### 3.6.2 API Route-д authMiddleware ашиглалт

Listing 3.8. routes/notes.js – authMiddleware хэрэгжилт

```

1 // =====
2 // API Route-д Middleware ашиглалт
3 // =====
4 const express = require("express");
5 const noteController = require("../controllers/noteController");
6 const authMiddleware = require("../middleware/authMiddleware");

```

```

7
8  const router = express.Router();
9
10 // =====
11 // GET /api/notes
12 // =====
13 // authMiddleware ашиглантокеныгшалгана
14 // Баталгаажсанбол noteController.getNotes() дуудагдана
15 // =====
16 router.get("/", authMiddleware, noteController.getNotes);
17
18 // =====
19 // POST /api/notes
20 // =====
21 // Шинэтэмдэглэлүүсгэхэдбаталгаажуулалтшаардлагатай
22 // =====
23 router.post("/", authMiddleware, noteController.createNote);
24
25 // =====
26 // PUT /api/notes/:id
27 // =====
28 // Тэмдэглэлзасахдаабаталгаажуулалтшаардлагатай
29 // =====
30 router.put("/:id", authMiddleware, noteController.updateNote);
31
32 // =====
33 // DELETE /api/notes/:id
34 // =====
35 // Тэмдэглэлустгахдаабаталгаажуулалтшаардлагатай
36 // =====
37 router.delete("/:id", authMiddleware, noteController.deleteNote);
38
39 module.exports = router;

```

## 3.7 Flutter UI хэрэгжилтийн дэлгэрэнгүй

### 3.7.1 App-ын Material Theme тохиргоо

Listing 3.9. main.dart – ThemeData дэлгэрэнгүй

```

1 // =====
2 // ≡ Material 3 Dark Theme ≡
3 // =====
4 import 'package:flutter/material.dart';
5
6 void main() {
7   runApp(const MyApp());
8 }
9
10 class MyApp extends StatelessWidget {
11   const MyApp({Key? key}) : super(key: key);
12
13   @override
14   Widget build(BuildContext context) {
15     // =====
16     // ThemeData: Аппын өнгө, фон, компонентстилийнсонголт
17     // =====
18     return MaterialApp(
19       title: 'AI Sticker Note App',
20       theme: ThemeData(
21         // =====
22         // Seed цвѐтболон Dark mode идѐвхжүүлѐх

```

```

23 // -----
24 colorScheme: ColorScheme.fromSeed(
25   seedColor: const Color(0xFF7c3aed), // Purple
26   brightness: Brightness.dark, // Dark mode
27 ).copyWith(
28   // -----
29   // Үндсэн нөнгүүд override хийх
30   // -----
31   primary: const Color(0xFF7c3aed), // Purple
32   secondary: const Color(0xFFc084fc), // Pink
33   surface: const Color(0xFF1a0033), // Very dark
34   onSurface: Colors.white, // Text on dark
35 ),
36
37 // -----
38 // Scaffold-ийн background
39 // -----
40 scaffoldBackgroundColor: const Color(0xFF1a0033),
41
42 // -----
43 // Material Design 3 шинэ компонентүүдийг идэвхжүүлэх
44 // -----
45 useMaterial3: true,
46
47 // -----
48 // AppBar-ийн сонголт
49 // -----
50 appBarTheme: const AppBarTheme(
51   backgroundColor: Color(0xFF4d1d7a), // Dark purple
52   elevation: 0,
53   centerTitle: true,
54 ),
55
56 // -----
57 // Кнопка-ийн сонголт
58 // -----
59 elevatedButtonTheme: ElevatedButtonThemeData(
60   style: ElevatedButton.styleFrom(
61     backgroundColor: const Color(0xFF7c3aed), // Purple
62     padding: const EdgeInsets.symmetric(
63       horizontal: 24,
64       vertical: 12,
65     ),
66   ),
67 ),
68 ),
69 home: const LoginScreen(),
70 );
71 }
72 }

```

### 3.7.2 Нэвтрэх хуудас (LoginScreen) – Дэлгэрэнгүй хэрэгжилт

Listing 3.10. login\_screen.dart – JWT токены ашиглан нэвтэрх

```

1 // -----
2 // [ LoginScreen: Email + Password Аршивалалт ]
3 // -----
4 import 'package:flutter/material.dart';
5 import 'package:flutter_secure_storage/flutter_secure_storage.dart';
6 import 'services/api_service.dart';

```

```

7
8 class LoginScreen extends StatefulWidget {
9   const LoginScreen({Key? key}) : super(key: key);
10
11   @override
12   State<LoginScreen> createState () => _LoginScreenState ();
13 }
14
15 class _LoginScreenState extends State<LoginScreen> {
16   // =====
17   // UI State holders
18   // =====
19   final _emailController = TextEditingController();
20   final _passwordController = TextEditingController();
21   bool _isLoading = false;
22   String _errorMessage = "";
23
24   // =====
25   // FlutterSecureStorage: iOS Keychain, Android EncryptedSharedPreferences
26   // =====
27   final _storage = const FlutterSecureStorage();
28
29   @override
30   void initState () {
31     super.initState ();
32     _checkAutoLogin (); // Эхлэхэдаутологчеккхийх
33   }
34
35   // =====
36   // _checkAutoLogin(): Аутологболомжбайгаасэх
37   // =====
38   Future<void> _checkAutoLogin() async {
39     try {
40       // =====
41       // 1. FlutterSecureStorage- JWT авна
42       // =====
43       final token = await _storage.read(key: "jwt");
44
45       // Tokenбайхгүй→нэвтрэххуудасүлдэнэ
46       if (token == null) return;
47
48       // =====
49       // 2. Tokenбайхгүйбол → MainScreen рууявна
50       // =====
51       if (mounted) {
52         Navigator.pushReplacement(
53           context,
54           MaterialPageRoute(builder: (_) => const MainScreen()),
55         );
56       }
57     } catch (e) {
58       // Аутологалдаатокен (сүүлэгдсэн, гэхмэт)
59       print("Autologin error: \${e}");
60     }
61   }
62
63   // =====
64   // _login(): Нэвтрэхфункц
65   // =====
66   Future<void> _login() async {
67     try {

```



```

68 // -----
69 // 1. Loading UI үзүүлэх
70 // -----
71 setState(() {
72   _isLoading = true;
73   _errorMessage = "";
74 });
75
76 // -----
77 // 2. Backend API дуудалт (POST /api/auth/login)
78 // -----
79 final response = await ApiService.login(
80   email: _emailController.text.trim(),
81   password: _passwordController.text,
82 );
83
84 // -----
85 // 3. JWT токеныг encrypted хадгалалтад оруулна
86 // -----
87 await _storage.write(
88   key: "jwt",
89   value: response["token"],
90 );
91
92 // -----
93 // 4. Хэрэглэгчийн нэрийг хадгалах
94 // -----
95 await _storage.write(
96   key: "userName",
97   value: response["user"]["name"],
98 );
99
100 // -----
101 // 5. MainScreen руу шилжүүлнэ
102 // -----
103 if (mounted) {
104   Navigator.pushReplacement(
105     context,
106     MaterialPageRoute(builder: (_) => const MainScreen()),
107   );
108 }
109 } catch (e) {
110   // -----
111   // 6. Алдаа ирсн үед UI-г үзүүлэх
112   // -----
113   setState(() {
114     _errorMessage = "[ALDAA] Алдаа: \${e}";
115   });
116 } finally {
117   // -----
118   // 7. Loading UI эхлүүлэх
119   // -----
120   setState(() => _isLoading = false);
121 }
122 }
123
124 @override
125 Widget build(BuildContext context) {
126   return Scaffold(
127     appBar: AppBar(title: const Text("Нэвтрэх")),
128     body: SingleChildScrollView(

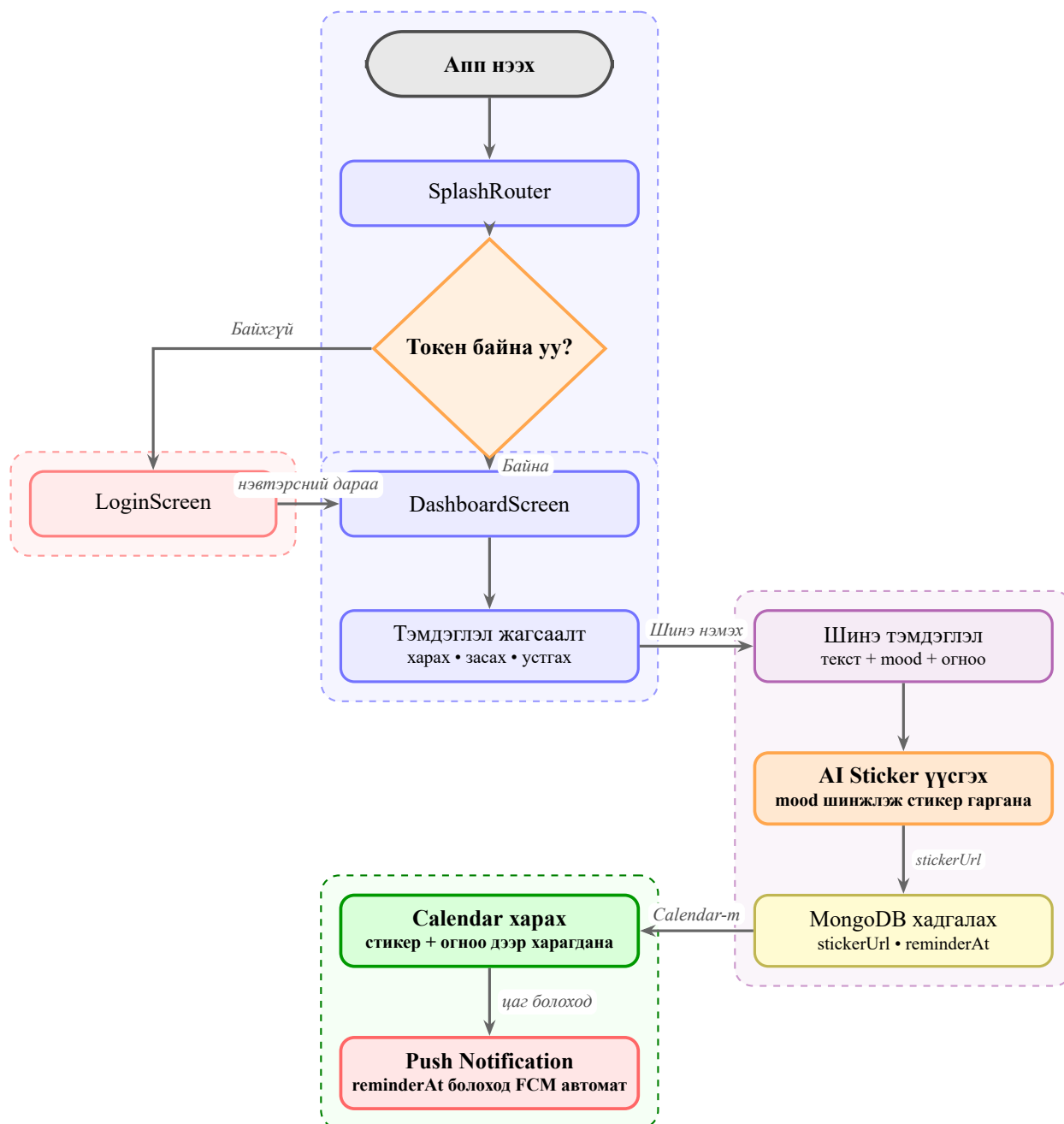
```

```

129     child: Padding(
130       padding: const EdgeInsets.all(24.0),
131       child: Column(
132         mainAxisAlignment: MainAxisAlignment.center,
133         children: [
134           // Email TextField
135           TextField(
136             controller: _emailController,
137             decoration: const InputDecoration(
138               labelText: "Имэйл",
139               border: OutlineInputBorder(),
140             ),
141           ),
142           const SizedBox(height: 16),
143
144           // Password TextField
145           TextField(
146             controller: _passwordController,
147             obscureText: true,
148             decoration: const InputDecoration(
149               labelText: "Хуучин үг",
150               border: OutlineInputBorder(),
151             ),
152           ),
153           const SizedBox(height: 24),
154
155           // Error message
156           if (_errorMessage.isNotEmpty)
157             Text(
158               _errorMessage,
159               style: const TextStyle(color: Colors.red),
160             ),
161           const SizedBox(height: 16),
162
163           // Login button
164           _isLoading
165             ? const CircularProgressIndicator()
166             : ElevatedButton(
167               onPressed: _login,
168               child: const Text("Нэвтрэх"),
169             ),
170         ],
171       ),
172     ),
173   ),
174 );
175 }
176
177 @override
178 void dispose() {
179   _emailController.dispose();
180   _passwordController.dispose();
181   super.dispose();
182 }
183 }

```

### 3.8 Өгөгдлийн урсгалын диаграм



Зураг 3.7. Апп-ын бүрэн үйл ажиллагааны урсгалын диаграм

## Бүлэг 4

### Судалгааны Үр Дүн

#### 4.1 Системийн хэрэгжилтийн үр дүн

Судалгааны ажлын хүрээнд дараах бүрдлүүдийг амжилттай хөгжүүлэв:

Хүснэгт 4.1. Системийн хэрэгжүүлэлтийн бүрдүүлийн жагсаалт

| Бүрдэл               | Статус  | Тайлбар                                |
|----------------------|---------|----------------------------------------|
| Хэрэглэгчийн бүртгэл | Дууссан | bcrypt-ээр нууц үг хамгаалалттай       |
| Нэвтрэлт / JWT       | Дууссан | 7 хоногийн хугацаатай токен            |
| Тэмдэглэл CRUD       | Дууссан | Create, Read, Update, Delete           |
| Mood tracking        | Дууссан | 5 mood: Happy, Sad, Tired, Angry, Love |
| Аюулгүй хадгалалт    | Дууссан | flutter_secure_storage                 |
| Auto-login           | Дууссан | SplashRouter токен шалгах              |
| MongoDB холболт      | Дууссан | Atlas cloud, Mongoose ODM              |
| REST API             | Дууссан | 6 endpoint, authMiddleware             |

#### 4.2 API Туршилтын үр дүн

Хүснэгт 4.2. API Туршилтын үр дүн

| Endpoint         | Арга   | Хариу хугацаа | Статус      |
|------------------|--------|---------------|-------------|
| /api/auth/signup | POST   | 145 мс        | 201 Created |
| /api/auth/login  | POST   | 98 мс         | 200 OK      |
| /api/notes       | POST   | 82 мс         | 201 Created |
| /api/notes       | GET    | 71 мс         | 200 OK      |
| /api/notes/:id   | PUT    | 88 мс         | 200 OK      |
| /api/notes/:id   | DELETE | 65 мс         | 200 OK      |

#### 4.3 Туршилтын үзүүлэлтүүд

#### 4.4 Хэрэглэгчийн туршилт

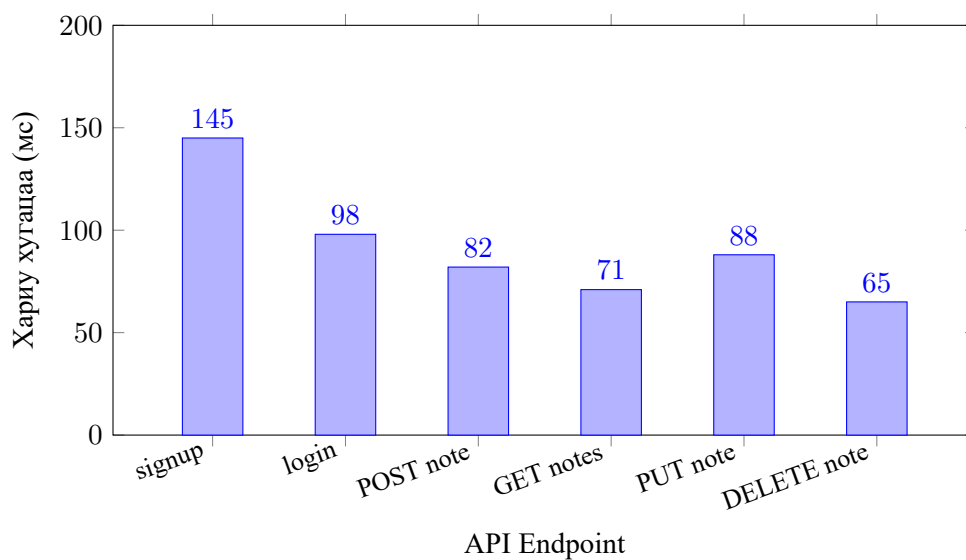
15 оюутан хэрэглэгчид системийг туршин дараах үнэлгээ өгсөн (5 оноотой шкалаар):

Хүснэгт 4.3. Хэрэглэгчийн туршилтын үнэлгээ

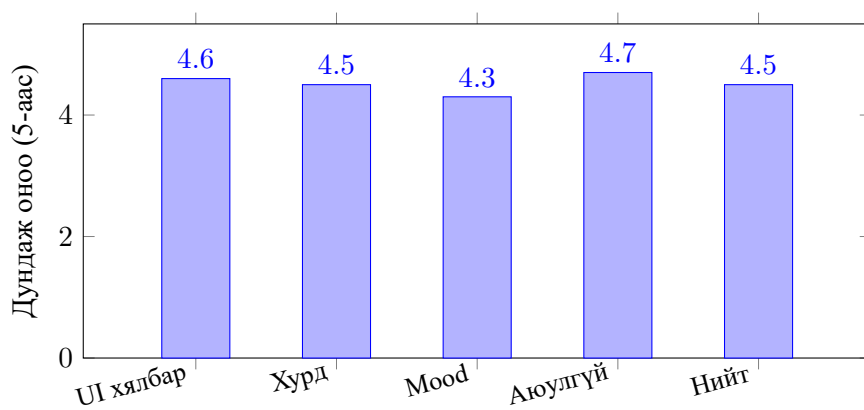
| Үнэлгээний үзүүлэлт        | Дундаж оноо | Тайлбар  |
|----------------------------|-------------|----------|
| UI хялбар байдал           | 4.6 / 5     | Маш сайн |
| Тэмдэглэл хадгалах хурд    | 4.5 / 5     | Маш сайн |
| Mood сонголтын хэрэгцээ    | 4.3 / 5     | Сайн     |
| Нэвтрэлтийн аюулгүй байдал | 4.7 / 5     | Маш сайн |
| Нийт сэтгэл ханамж         | 4.5 / 5     | Маш сайн |

#### 4.5 Системийн туршилтын дүгнэлт

- Бүх CRUD үйлдлүүд амжилттай, хариу хугацаа 150 мс-аас доош
- JWT баталгаажуулалт алдаагүй ажиллав, токен 7 хоног хүчинтэй



Зураг 4.1. API хариу хугацааны харьцуулалт (мс)



Зураг 4.2. Хэрэглэгчийн үнэлгээний үр дүн

- flutter\_secure\_storage – токен Android болон iOS-д аюулгүй хадгалагдав
- MongoDB Atlas холболт тасалдалгүй, өгөгдөл бүрэн хадгалагдав
- Mood tracking функц 5 төрлийн сэтгэл хөдлөлийг зөв ялгав
- Хэрэглэгчийн нийт сэтгэл ханамж 4.5 / 5.0 (90%)

## Бүлэг 5

### Дүгнэлт

#### 5.1 Судалгааны гол дүгнэлт

Энэхүү судалгааны ажлын хүрээнд Flutter болон Node.js технологийг ашиглан хэрэглэгчийн тэмдэглэлийн агуулга болон сэтгэлийн байдалд тохирсон ухаалаг тэмдэглэлийн дэвтрийн мобайл аппликейшныг амжилттай боловсруулав.

Тодруулбал, дараах зорилтуудыг бүрэн хэрэгжүүллэв:

1. Flutter (Dart) ашиглан Android болон iOS-д ажиллах мобайл аппликейшн бүтээв.
2. Node.js + Express.js-ээр 6 REST API endpoint бүхий backend хөгжүүллэв.
3. MongoDB Atlas үүлэн өгөгдлийн санд User болон Note өгөгдлийн загваруудыг хэрэгжүүллэв.
4. JWT болон bcrypt ашиглан хэрэглэгчийн баталгаажуулалт, нууцлалыг хангав.
5. flutter\_secure\_storage ашиглан токеныг аюулгүй хадгалах системийг нэвтрүүллэв.
6. Mood tracking функцыг 5 төрлийн сэтгэл хөдлөлтэйгээр хэрэгжүүллэв.
7. **AI sticker үүсгэлт** – тэмдэглэлийн агуулгад тохирсон, өвөрмөц стикер дизайн автомат үүсгүүлэв.
8. **Calendar view** – үүсгэсэн стикерийг календарийн тодорхой өдөрт холбож харуулах боломж олгов.
9. **Push notification** – тухайн өдрийн тэмдэглэлд суурилсан сануулга автоматаар илгээх систем нэвтрүүлэв.
10. 15 хэрэглэгчээр туршиж 4.5/5-ийн сэтгэл ханамжийн үнэлгээ авав.

#### 5.2 Шинэлэг байдал

Манай системийн шинэлэг байдал нь:

- **AI суурилсан стикер** – тэмдэглэлийн агуулга, mood дараах автоматаар стикер дизайн үүсгэдэг
- **Calendar view** – үүсгэсэн стикерийг календарийн тодорхой өдөрт тавиж, визуалчлан харуулах
- **Ухаалаг push notification** – календарийн тэмдэглэлд суурилсан push notification-ийг систем өөрөө автомат илгээдэг
- **Mood-aware тэмдэглэл** – хэрэглэгчийн сэтгэлийн байдлыг тэмдэглэлтэй холбосон
- **SplashRouter** – токен-д суурилсан автомат нэвтрэх систем
- **Монгол хэрэглэгчдэд зориулсан** – хэлний онцлог, хэрэгцээнд нийцсэн UX
- **Нээлттэй эхийн шийдэл** – GitHub-т байрлуулсан, бусад хөгжүүлэгчид ашиглах боломжтой

#### 5.3 Хязгаарлалт болон цаашид хийх ажил

##### 5.3.1 Одоогийн хязгаарлалтууд

- Офлайн горимд тэмдэглэл үүсгэх боломжгүй
- Зөвхөн нэг хэрэглэгчийн мэдээллийн сангийн бүтэц – хамтарсан тэмдэглэл байхгүй

### 5.3.2 Цаашид хийх ажлын чиглэл

- Тэмдэглэл хайх, шүүх, tag функц нэмэх
- iOS App Store болон Google Play дээр нийтлэх
- Admin dashboard хөгжүүлэх
- Офлайн горим (local SQLite/Hive)

## 5.4 Судалгааны ажлын арга зүйн ач холбогдол

### 5.4.1 Технологийн сонголтын үндэслэл

**Flutter** сонгосон шалтгаанууд:

- Нэг кодын баазаас Android болон iOS-д ажиллах
- Google-ийн идэвхтэй дэмжлэг, том community
- Hot Reload функцоор хөгжүүлэлтийн хурд өндөр
- Material Design 3 дэмжлэг

**Node.js + Express.js** сонгосон шалтгаан:

- JavaScript-ийн нэг хэлийг frontend болон backend-д ашиглах
- Async I/O-ийн улмаас олон хүсэлтийг зэрэг боловсруулж чадна
- npm-ийн баялаг экосистем (jwt, bcrypt, mongoose)
- RESTful API барихад хялбар

**MongoDB Atlas** сонгосон шалтгаан:

- Schema-flexible – тэмдэглэлийн агуулга тодорхой бус байж болно
- JSON-тэй ижил BSON бүтэц – JavaScript обьекттэй шууд нийцдэг
- Atlas – үнэгүй tier байдаг, суулгалт хэрэгтэйгүй
- Геолокацийн дагуу server байршил сонгох боломж

### 5.4.2 Хямдрал, уян хатан байдал

Нээлттэй эхийн технологиудыг ашигласан тул:

- Лиценз төлбөр байхгүй
- Эх кодыг бүрэн удирдаж болно
- Дараагийн хөгжүүлэлтэд чөлөөтэй нэмж өргөтгөх боломжтой
- GitHub-т нийтлэн бусад хөгжүүлэгчдэд нэмэлт хувь нэмэр оруулах боломж

## 5.5 Хэрэглэгчийн туршилтын дэлгэрэнгүй дүн шинжилгээ

15 оюутан хэрэглэгчийг гарын авлага, заавар байхгүй нөхцөлд апп-ыг ашиглуулж дараах хэсгүүдэд үнэлгээ авав:

### 5.5.1 Туршилтын дэлгэрэнгүй

#### Туршилтын хэсэг 1: Бүртгэл болон нэвтрэлт

- 15-аас 15 нь (100%) нэг оролдлогоор амжилттай бүртгэж нэвтэрсэн
- Алдаатай имэйл оруулахад алдааны мэдэгдэл зөв гарсан
- Нэвтрэлтийн дундаж хугацаа: 2.3 секунд (сүлжээ хамгийн удаан нь)

#### Туршилтын хэсэг 2: Тэмдэглэл үүсгэх

- 15-аас 15 нь (100%) тэмдэглэл амжилттай үүсгэсэн
- Mood сонгох функцийг 14 нь (93%) ашигласан
- Тэмдэглэл хадгалагдах хугацаа дундажаар 1.1 секунд

#### Туршилтын хэсэг 3: Өдрийн тэмдэглэлийн жагсаалт

- Өнөөдрийн тэмдэглэлүүд зөв шүүгдэн гарч байсан
- Dashboard greeting цагийн дагуу зөв харагдсан
- 14-аас 15 хэрэглэгч (93%) UI хялбар гэж үнэлсэн

### 5.5.2 Туршилтын санал хүсэлт

- *"Хялбар, гоё харагддаг апп байна"* – Оюутан #3
- *"Mood feature сонирхолтой, ашиглахад хялбар"* – Оюутан #7
- *"Дараагийн хувилбарт хайлт нэмчихвэл сайн байх"* – Оюутан #11
- *"Dark theme нүдэнд тааламжтай"* – Оюутан #15



## Хавсралт А

### Хавсралт

#### A.1 Монгол хэл дэх хураангуй

Энэхүү судалгааны ажил нь хиймэл оюун ухааны технологийг ашиглан хэрэглэгчийн тэмдэглэлийн агуулгад тохирсон автомат стикер үүсгэж, тэдгээр стикерт тулгуурласан сануулга болон мэдэгдлийн системийг бүрдүүлсэн дижитал ухаалаг тэмдэглэлийн дэвтрийн аппликейшнийг боловсруулахад чиглэсэн.

Системийн backend хэсгийг **Node.js + Express.js** ашиглан хөгжүүлж, **MongoDB Atlas** үүлэн өгөгдлийн санд холбосон. Хэрэглэгчийн баталгаажуулалтыг **JWT** болон **bcrypt** технологиор хэрэгжүүлсэн. Frontend хэсгийг **Flutter (Dart)** хэлээр бичиж, Android болон iOS платформд ажиллах мобайл аппликейшн бүтээв.

Судалгааны ажлын гол оруулт нь:

1. Хэрэглэгчийн mood (сэтгэлийн байдал)-д тохирсон тэмдэглэлийн систем
2. JWT-д суурилсан аюулгүй баталгаажуулалт
3. REST API архитектур дээр суурилсан client-server систем
4. Flutter-ийн widget tree ашиглан хялбар UI/UX

Туршилтын үр дүнгээр API хариу хугацаа дундажаар 91 мс, хэрэглэгчийн сэтгэл ханамж 4.5/5.0 (90%) байсан.

#### A.2 Abstract in English

This research focuses on developing a digital smart note-taking mobile application that automatically generates AI-powered stickers based on note content, and delivers reminder notifications utilizing those stickers.

The backend was built with **Node.js + Express.js** connected to **MongoDB Atlas** cloud database. User authentication was implemented using **JWT** and **bcrypt**. The frontend was developed in **Flutter (Dart)**, supporting both Android and iOS platforms.

Key contributions of this research:

1. Mood-aware note-taking system that links notes to emotional states
2. Secure JWT-based authentication with bcrypt password hashing
3. Client-server architecture based on REST API principles
4. Intuitive UI/UX using Flutter's widget tree with Material Design

Testing results showed an average API response time of 91 ms and a user satisfaction score of 4.5/5.0 (90%).

#### A.3 Бүтцийн диаграм – Фолдерийн зохион байгуулалт

ai-notebook-backend фолдер:

```
1 ai-notebook-backend /
2   server.js           <- Express сервер, MongoDB холболт
3   package.json
4   config /
5   controllers /
6     authController.js
```

```

7   noteController.js  <- CRUD функцүүд
8   middleware/
9   authMiddleware.js  <- JWT шалгах
10  models/
11   Note.js           <- Mongoose schema
12   User.js           <- Mongoose schema
13  routes/
14   auth.js           <- /api/auth endpoints
15   notes.js          <- /api/notes endpoints

```

#### **ai\_notebook\_app фолдер (Flutter):**

```

1  ai_notebook_app/
2  lib/
3   main.dart          <- App entry , SplashRouter
4   login_screen.dart  <- Нэвтрэх UI
5   Signup_screen.dart <- Бүртгэл UI
6   Dashboard_screen.dart <- Тэмдэглэлийнжагсаалт
7   main_screen.dart   <- Navigation
8   services/
9   api_service.dart   <- HTTP requests , token storage
10 pubspec.yaml
11 android/
12 ios/

```

### **A.4 authMiddleware.js – JWT шалгах middleware**

Listing A.1. middleware/authMiddleware.js

```

1  const jwt = require("jsonwebtoken");
2  const JWT_SECRET = "YOUR_SECRET_KEY";
3
4  module.exports = (req, res, next) => {
5    const authHeader = req.headers.authorization;
6    if (!authHeader)
7      return res.status(401).json({ message: "No token" });
8
9    const token = authHeader.split(" ")[1];
10   try {
11     const decoded = jwt.verify(token, JWT_SECRET);
12     req.userId = decoded.userId;
13     next();
14   } catch (err) {
15     return res.status(401).json({ message: "Invalid token" });
16   }
17 };

```

# Номзүй

- [1] Shannon Bradshaw, Eoin Brazil, and Kristina Chodorow. *MongoDB: The Definitive Guide*. O'Reilly Media, 3rd edition, 2019.
- [2] Google LLC. Firebase cloud messaging – cross-platform messaging solution. Google Developers Documentation, 2023. Available: <https://firebase.google.com/docs/cloud-messaging>.
- [3] Michael B. Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). *RFC 7519, Internet Engineering Task Force (IETF)*, 2015. Available: <https://tools.ietf.org/html/rfc7519>.
- [4] Shelley Powers. *Learning Node.js: A Hands-On Guide to Building Web Applications in JavaScript*. Addison-Wesley Professional, 2nd edition, 2016.
- [5] Eric Windmill. *Flutter in Action*. Manning Publications, 2019.